



Módulo de IA Generativa

Bloque II



En que consiste y como está estructurado el módulo

En este módulo aprenderás a dominar los fundamentos, herramientas y aplicaciones prácticas de la IA Generativa. Desde cómo funcionan los grandes modelos de lenguaje (LLMs) hasta cómo integrarlos en productos reales con responsabilidad y criterio.

- 1

Introducción a LLMs e integración con APIs
Visión general de los modelos de lenguaje (LLMs), cómo funcionan a nivel conceptual y cómo integrarlos en aplicaciones reales mediante APIs para construir soluciones basadas en IA.
- 2

Ingeniería de Prompts
Aprenderás a diseñar prompts efectivos para obtener resultados precisos y útiles, aplicando técnicas avanzadas como prompting estructurado, few-shot y control de comportamiento del modelo.
- 3

RAG – Retrieval Augmented Generation
Exploración del patrón RAG para conectar LLMs con fuentes de datos externas, mejorando la precisión y contextualización de las respuestas mediante recuperación de información.
- 4

Pipelines ML + GenAI
Construcción de pipelines que combinan modelos de Machine Learning tradicionales con IA generativa, integrando procesamiento de datos, inferencia y generación de contenido.
- 5

Tutoría y ejercicios prácticos
Sesión enfocada en resolver dudas y aplicar lo aprendido mediante ejercicios guiados, reforzando conceptos clave con casos reales.
- 6

Agentes de IA: arquitectura, herramientas y memoria
Introducción a los agentes autónomos: cómo se estructuran, qué componentes los forman (herramientas, memoria, planificación) y cómo diseñarlos.
- 7

Frameworks de Agentes: LangChain y LangGraph
Uso práctico de frameworks populares para construir agentes, orquestar flujos y gestionar estados complejos en aplicaciones con IA.
- 8

Agentes en AWS Bedrock
Despliegue y gestión de agentes utilizando AWS Bedrock, integrando modelos fundacionales con servicios cloud para soluciones escalables.
- 9

Orquestación y despliegue de agentes + proyecto
Diseño completo de sistemas con agentes: orquestación, despliegue en producción y desarrollo de un proyecto final integrando todos los conceptos del curso.
- 10

Tutoría y resolución de dudas del proyecto
Espacio para resolver dudas, revisar avances y recibir orientación personalizada para mejorar el proyecto.



RAG – Retrieval Augmented Generation

Ajuste fino de modelos, uso de RAG para acceso a información, consideraciones de seguridad, sesgos, privacidad y marcos regulatorios para un uso responsable.

1

Fine-tuning y RAG

Exploraremos cómo ajustar modelos preentrenados para tareas específicas mediante fine-tuning y cómo mejorar la precisión de las respuestas combinando LLMs con búsqueda de información (RAG).

2

Seguridad en IA Generativa

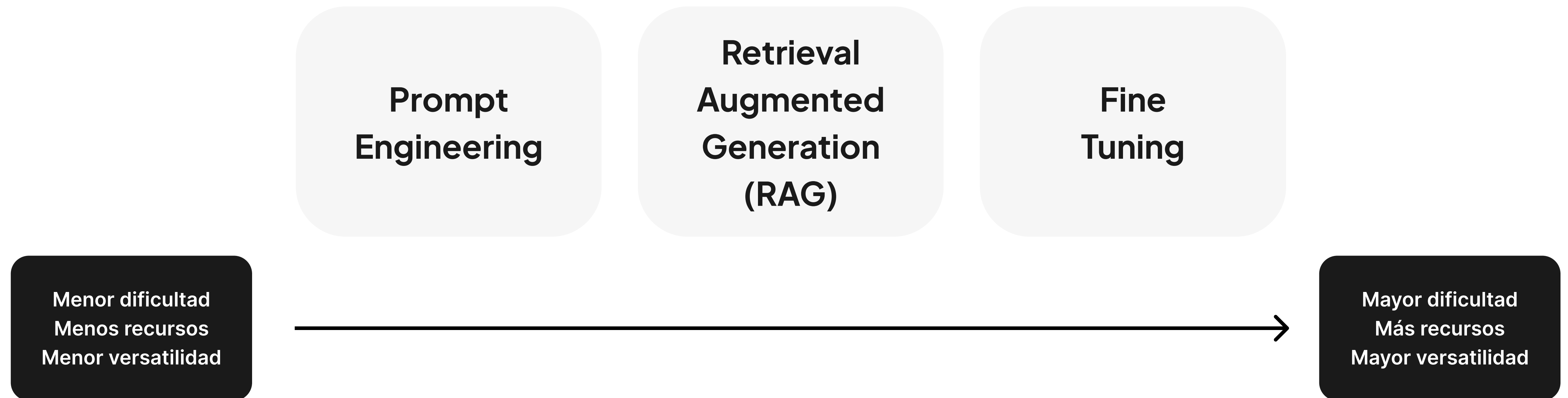
Analizaremos los principales riesgos de seguridad en IA generativa, como ataques y filtraciones, y aprenderemos buenas prácticas para proteger los datos y garantizar la privacidad.



Introducción al ajuste de modelos

Los modelos fundacionales destacan por su gran flexibilidad y capacidad para comprender el lenguaje humano de manera amplia. No obstante, cuando se enfrentan a tareas muy específicas, pueden presentar limitaciones o no ofrecer resultados completos, debido a que su rendimiento depende de los datos con los que fueron entrenados y pueden estancarse en ciertas funciones particulares.

Por ello, es necesario ajustar el modelo para que se adecue a los requerimientos particulares de cada caso:



Prompt engineering

Se trata de diseñar y modificar el texto de entrada al modelo para orientarlo, a través de contexto, indicaciones o ejemplos, con el fin de obtener una respuesta (salida) que sea lo más precisa y adecuada posible.

Pregunta: ¿Qué es un girasol?

Respuesta: Un girasol es una planta que destaca por su gran flor amarilla.



Contexto: Estás en un taller de jardinería y el instructor pregunta sobre plantas comunes.

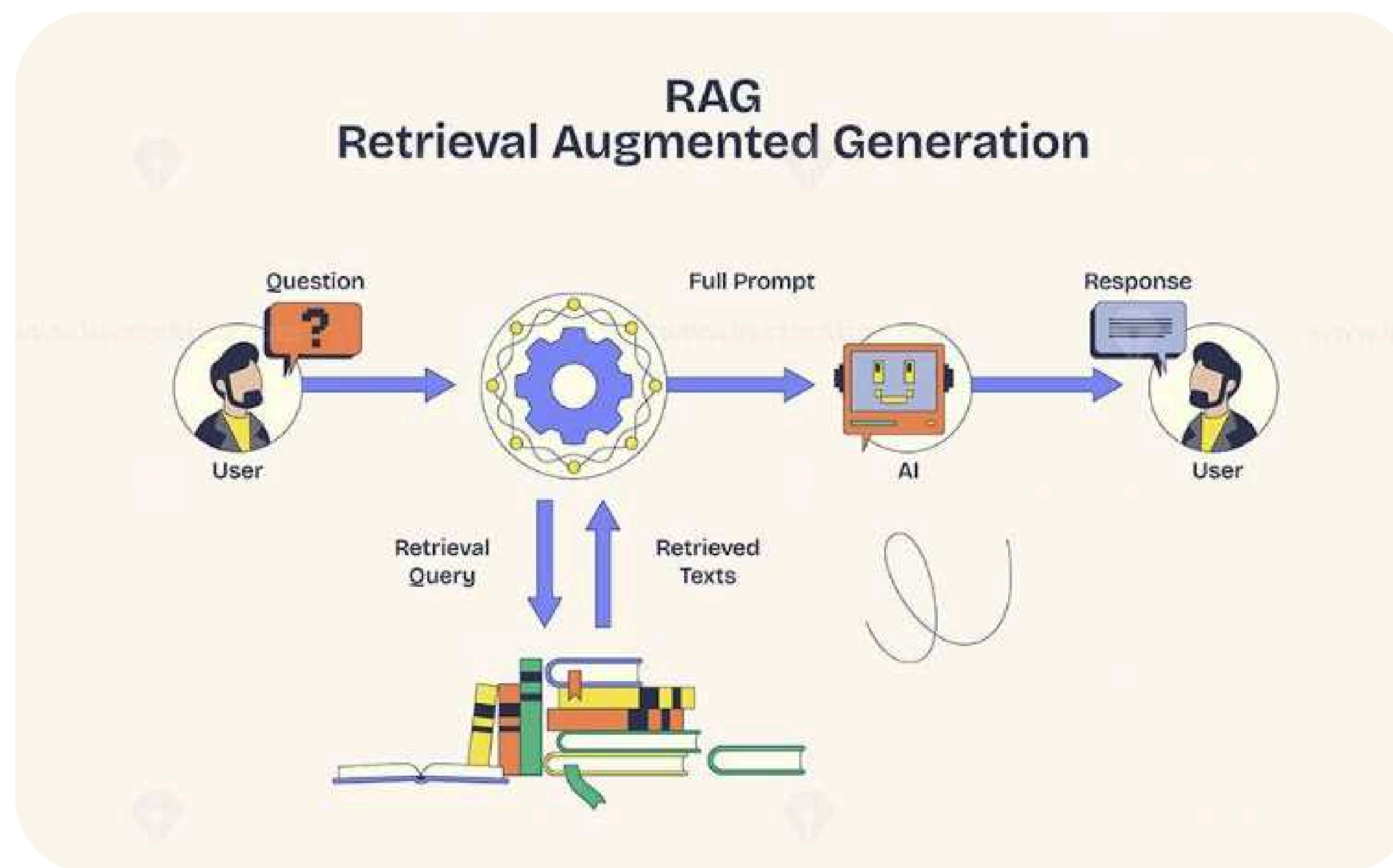
Pregunta: ¿Qué es un girasol?

Respuesta: El girasol es una planta muy apreciada tanto por su belleza como por sus semillas, que se utilizan en la alimentación y para extraer aceite.



Limitaciones de los modelos anteriores

Permite que un LLM busque la respuesta empleando un contexto privado de documentación.





Fine Tuning

El ajuste fino consiste en modificar y optimizar los parámetros de un modelo ya entrenado.

Normalmente, se reentrenan las capas superiores de la red neuronal utilizando nuevos datos que están correctamente etiquetados.

Esta técnica es especialmente útil cuando se requiere que un modelo de lenguaje grande (LLM) realice tareas muy especializadas, como por ejemplo un LLM experto en normativas bancarias, o cuando es necesario que se adapte eficazmente a múltiples contextos, como ocurre con un chatbot para atención al cliente.



Resumen

No hay una estrategia única que funcione mejor en todos los casos; la elección depende de las particularidades del caso de uso.

Muchas veces, las técnicas pueden combinarse para obtener mejores resultados.

Asimismo, no existe una plataforma, arquitectura o tipo de modelo que sea superior de forma absoluta, sino que todo depende de las necesidades específicas de cada situación.



Introducción al RAG

La arquitectura RAG fue creada por Meta (antes conocida como Facebook) en 2020, tal como se presenta en su artículo titulado “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”.

https://ai.meta.com/RAG_01_v5/videos/244800523626272/?idorvanity=712362492679853



Introducción al RAG II

Como hemos visto, un LLM puede incorporar nuevos conocimientos a través del fine-tuning, que implica actualizar ciertos parámetros (pesos) del modelo.

Sin embargo, este proceso suele ser costoso y lento.

Además, no resulta eficiente si nuestra base de datos está en constante cambio, con actualizaciones frecuentes de información.

Otro punto importante es que, aunque el LLM pueda “saber” algo, no siempre podemos rastrear de dónde obtuvo esa información, lo que incrementa el riesgo de que genere respuestas erróneas o “alucinaciones”.

Por último, cualquier persona con acceso al modelo podría potencialmente consultar esa información.



Introducción al RAG III

Las arquitecturas RAG funcionan mediante un proceso de dos etapas:

1

Recuperación (Retrieval)

Se emplea un motor de búsqueda para obtener fragmentos relevantes de una base de datos o de un corpus vectorial.

2

Generación (Generation)

Un modelo generativo, típicamente un LLM, utiliza la información recuperada para producir respuestas.



Retrieval Augmented Generation

El proceso sigue estos pasos

- 1

Entrada del usuario

Se recibe una consulta o prompt enviado por el usuario.
- 2

Búsqueda en el repositorio

Aprenderás a diseñar prompts efectivos para obtener resultados precisos y útiles, aplicando técnicas avanzadas como prompting estructurado, few-shot y control de comportamiento del modelo.
- ✓

La consulta se convierte en un embedding mediante técnicas de representación vectorial.

Se lleva a cabo una búsqueda sobre la base de datos vectorial, que puede realizarse de distintas formas; una común es usando la similitud del coseno.

Con esta medida de similitud, se identifican los fragmentos de texto más cercanos dentro de diferentes documentos.
- 3

Integración de la información

Se eligen los top-N documentos o fragmentos con mayor puntuación de similitud y se integran al prompt original, aportando un contexto específico.
- 4

Generación de la respuesta

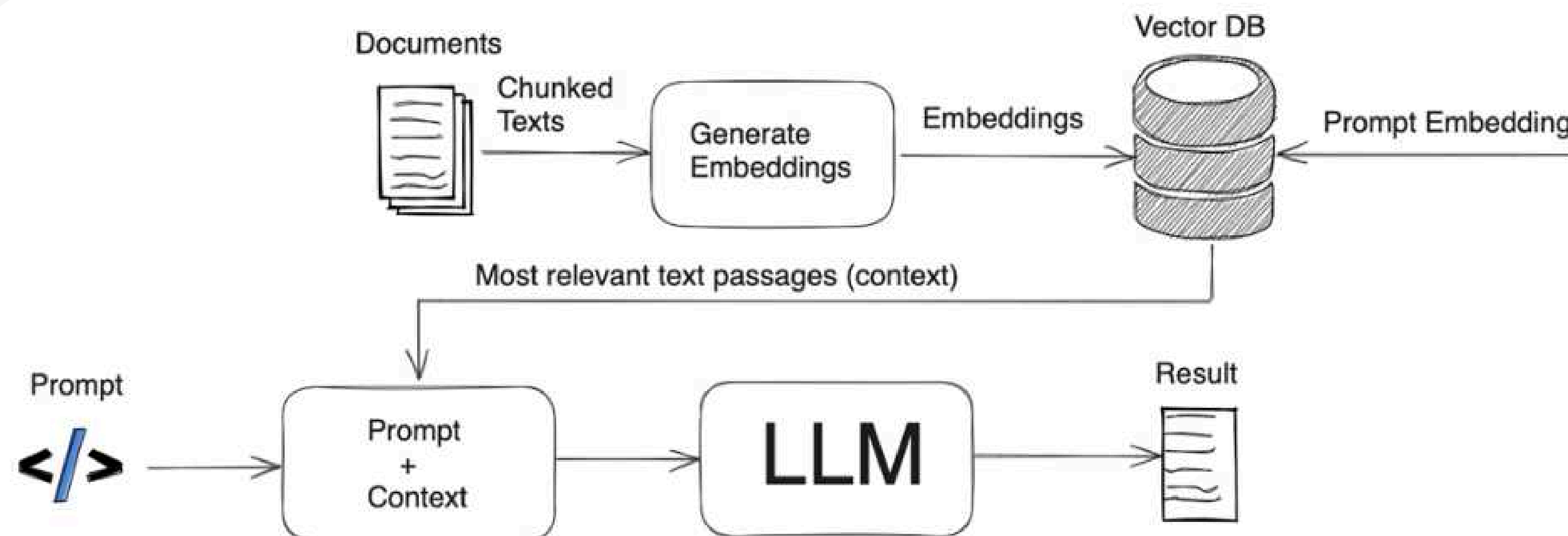
El modelo generativo crea una respuesta enriquecida que aprovecha tanto la consulta como la información externa incorporada.
- 5

Trazabilidad de referencias

Junto con la respuesta, se pueden incluir las referencias a los fragmentos y documentos utilizados para fundamentar la respuesta final.



Retrieval Augmented Generation





Retrieval Augmented Generation

El chunking en el proceso de RAG (Retrieval-Augmented Generation) es un paso clave en la preparación de datos para la búsqueda y generación de información relevante. En términos simples, se refiere a la técnica de dividir grandes volúmenes de texto o datos en fragmentos más pequeños y manejables (o chunks) antes de que sean utilizados en el modelo de RAG.

¿Por qué es importante el chunking en RAG?

1

Mejora de la eficiencia

Los modelos de RAG necesitan manejar grandes volúmenes de texto (como artículos, libros, bases de datos, etc.). Si se alimenta el modelo con documentos enteros, esto podría ser ineficiente y generar respuestas menos precisas. Al fragmentar el texto en partes más pequeñas, la búsqueda se vuelve más rápida y precisa.

2

Precisión en la recuperación

Al dividir el texto en fragmentos, el modelo puede recuperar piezas de información más específicas y relevantes. Esto es especialmente útil cuando la información es muy detallada o variada, porque el modelo puede identificar exactamente cuál chunk es el más adecuado para una pregunta dada.



Estrategias para prevenir errores

La indexación vectorial consiste en convertir la información de lenguaje natural en representaciones numéricas de alta dimensión, es decir, en vectores. Estos vectores son generados mediante modelos de embeddings (como Word2Vec, GloVe o modelos más avanzados como BERT y sus variantes), que capturan el significado semántico de los chunks de texto. De esta forma, cada chunk de documento se transforma en un vector de características que refleja sus contenidos y contexto de manera densa y matemática.

Embeddings

- ✓ Se utilizan modelos de embeddings para convertir cada fragmento de texto (chunk) en un vector. Estos vectores son representaciones densas y numéricas que permiten comparar fácilmente el contenido semántico de los fragmentos.

Revisión manual

- ✓ Los vectores generados por los embeddings se almacenan en una base de datos optimizada para búsqueda vectorial, como FAISS, Pinecone o Elasticsearch.

Uso como borrador, no como fuente definitiva

- ✓ Además de almacenar los vectores, la indexación vectorial puede incluir metadatos asociados a cada fragmento. Los metadatos pueden incluir información adicional como el origen del texto, fecha de publicación, categorías o cualquier otra información relevante. Esto permite realizar búsquedas más sofisticadas, no solo por el contenido semántico, sino también aplicando filtros basados en estos metadatos, lo que mejora la precisión y la trazabilidad de las respuestas generadas.



Retrieval Augmented Generation – Búsqueda de información

La búsqueda de la información se lleva a cabo en varias fases, que incluyen tanto la recuperación de documentos o fragmentos relevantes como el filtrado de esos resultados para que sean lo más útiles posible en el contexto de la consulta.

Consulta

- ✓ El primer paso es cuando el usuario o sistema formula una consulta, generalmente en lenguaje natural, que puede contener palabras clave, preguntas específicas o descripciones vagas del tema de interés. El objetivo es recuperar documentos o fragmentos que sean semánticamente relevantes para la consulta.

Conversión de la consulta en un vector

- ✓ Antes de realizar la búsqueda, la consulta también es convertida en un vector de características mediante un modelo de embedding, similar al proceso realizado para los chunks de texto en la indexación.

Búsqueda en el índice vectorial

- ✓ Una vez que tanto la consulta como los chunks de texto (o documentos) están representados en forma de vectores, la siguiente etapa es buscar los vectores más cercanos en la base de datos o índice vectorial. Existen diferentes métricas para medir la proximidad entre los vectores, como:

1

Distancia Euclidiana

Mide la distancia lineal entre dos puntos en un espacio multidimensional.

2

Similitud del coseno

Mide el ángulo entre dos vectores, lo cual es útil para ver qué tan similares son en términos de dirección, sin tener en cuenta su magnitud.

La búsqueda basada en vectores permite identificar fragmentos de texto que son semánticamente similares a la consulta, incluso si no contienen las mismas palabras exactas.



Retrieval Augmented Generation – Búsqueda de información

La búsqueda de la información se lleva a cabo en varias fases, que incluyen tanto la recuperación de documentos o fragmentos relevantes como el filtrado de esos resultados para que sean lo más útiles posible en el contexto de la consulta.

Recuperación de los fragmentos más relevantes

- ✓ El sistema recupera los chunks o documentos que son más cercanos a la consulta. En el caso de RAG, se suelen recuperar varios fragmentos para generar una respuesta más completa y matizada. Estos fragmentos pueden ser párrafos, oraciones, o incluso más detalles contextuales, dependiendo de cómo se haya realizado el chunking.

Filtrado y refinamiento (opcional)

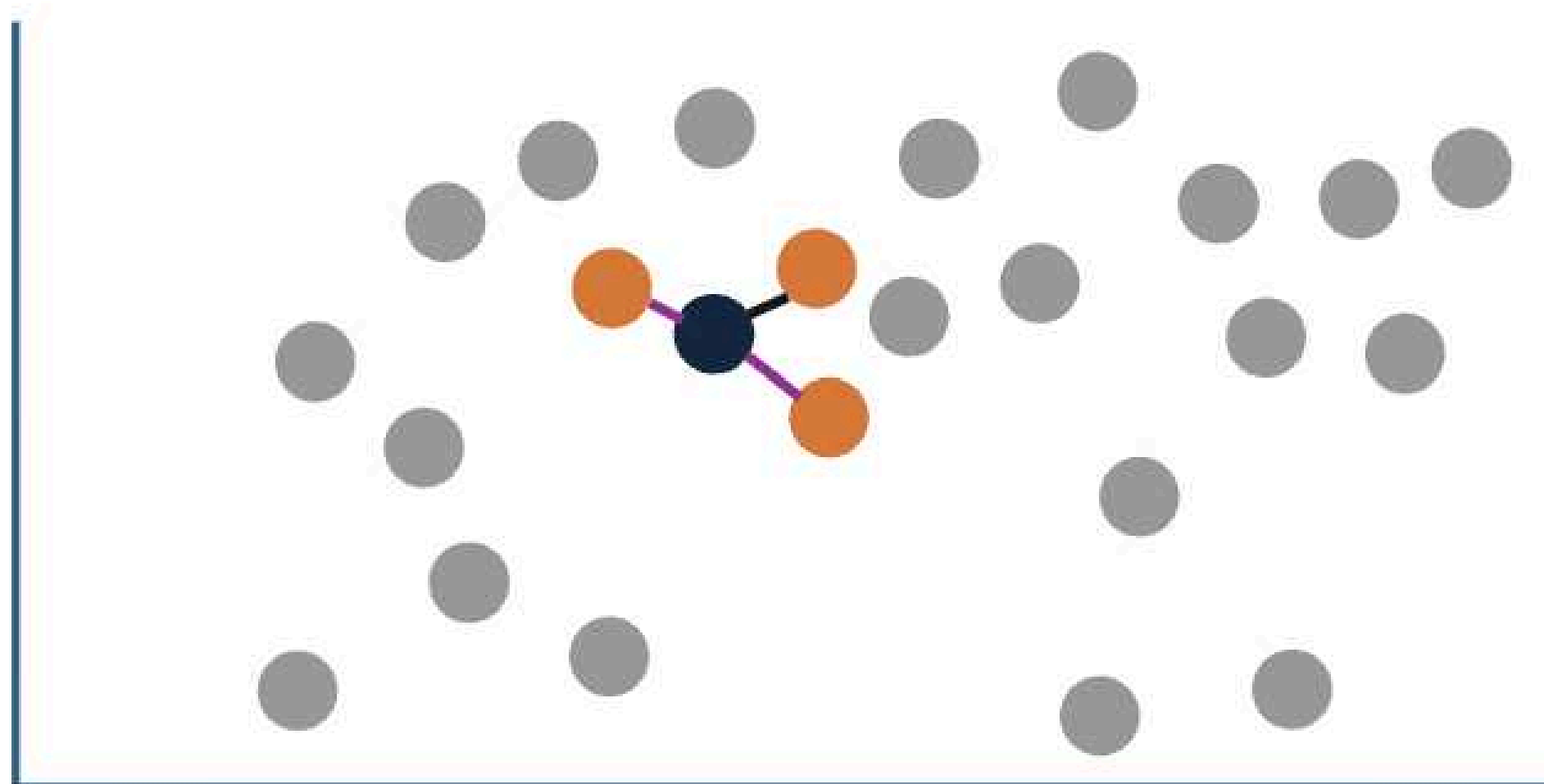
- ✓ En algunos casos, después de la búsqueda inicial, puede haber una etapa de filtrado que permita refinar los resultados con base en ciertos metadatos. Por ejemplo, si se está buscando información sobre un tema reciente, el sistema puede filtrar solo aquellos fragmentos que están fechados en los últimos meses. También puede haber ajustes de relevancia o aplicar ciertos criterios adicionales para ajustar los resultados.



Limitaciones de los modelos anteriores

Permite que un LLM busque la respuesta empleando un contexto privado de documentación.

TopN = 3
Min = 0.9*



● Query
● Chunks



Retrieval Augmented Generation – Generación de contenido

Una vez que se tiene la información relevante, el siguiente paso es generar la respuesta propiamente dicha. Este proceso se lleva a cabo utilizando un modelo generativo, como GPT o T5, que tiene la capacidad de generar texto de manera fluida y coherente. Sin embargo, a diferencia de los modelos generativos tradicionales, que solo se basan en su conocimiento previo, los modelos en RAG incorporan la información recuperada.

Entrada al modelo generativo

- ✓ La información recuperada se introduce como entrada al modelo generativo, junto con la consulta original. El sistema puede pasar tanto la consulta como los fragmentos de información recuperados de una manera estructurada o desordenada, dependiendo de cómo se haya diseñado el flujo de trabajo.

Contextualización y fusión de la información

- ✓ El modelo generativo no solo utiliza los chunks como referencia, sino que los contextualiza con el objetivo de construir una respuesta coherente y fluida. Esto puede implicar:

1

Fusionar información

Mide la distancia lineal entre dos puntos en un espacio multidimensional.

2

Elaborar

Una respuesta que esté alineada con el contexto de la pregunta, incluso si la respuesta está implícita en varios fragmentos. Este paso es clave porque el modelo debe generar una respuesta coherente, fluida y contextualmente adecuada usando la información de los chunks recuperados, sin que se sientan como fragmentos desconectados.

Generación del texto

- ✓ En la generación de la respuesta, el modelo generativo utiliza su capacidad para predecir la siguiente palabra o secuencia de palabras de acuerdo con el contexto y la información proporcionada. El resultado es un texto que responde de manera precisa a la consulta del usuario, utilizando la información recuperada en su estructura.



Retrieval Augmented Generation – Evaluación del modelo

En la fase de recuperación de información, el objetivo es asegurarse de que el modelo sea capaz de recuperar fragmentos de texto relevantes de la base de datos vectorial. Aquí hay algunas métricas clave:

Precisión (Precision)

- ✓ Mide el porcentaje de los fragmentos recuperados que son realmente relevantes para la consulta. Si el modelo recupera 10 fragmentos y solo 6 son relevantes, la precisión es

$$\text{Precisión} = \frac{\text{Fragmentos Relevantes Recuperados}}{\text{Total de Fragmentos Recuperados}}$$

Recall

- ✓ Mide el porcentaje de los fragmentos relevantes que han sido recuperados. Si hay 10 fragmentos relevantes en total y el modelo recupera 6 de ellos, el recall es

$$\text{Recall} = \frac{\text{Fragmentos Relevantes Recuperados}}{\text{Total de Fragmentos Relevantes}}$$

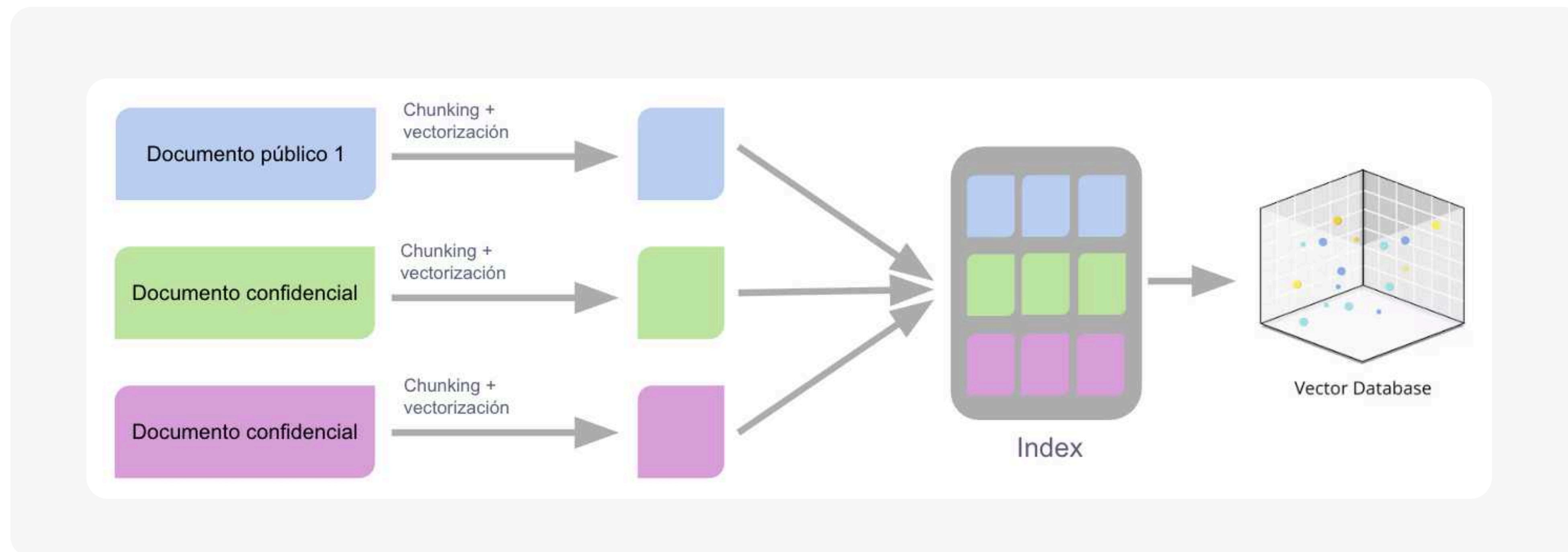


Seguridad en IA Generativa



Control de acceso a la información de contexto

En RAG, al igual que en otras arquitecturas, es fundamental contar con una capa de seguridad para proteger la información. Esto resulta especialmente desafiante debido al desacople entre el acceso al índice vectorial y a los documentos originales que lo alimentan. Como consecuencia, un usuario sin permiso para consultar ciertos documentos podría, indirectamente, obtener información sobre ellos a través de las respuestas generadas por el LLM.

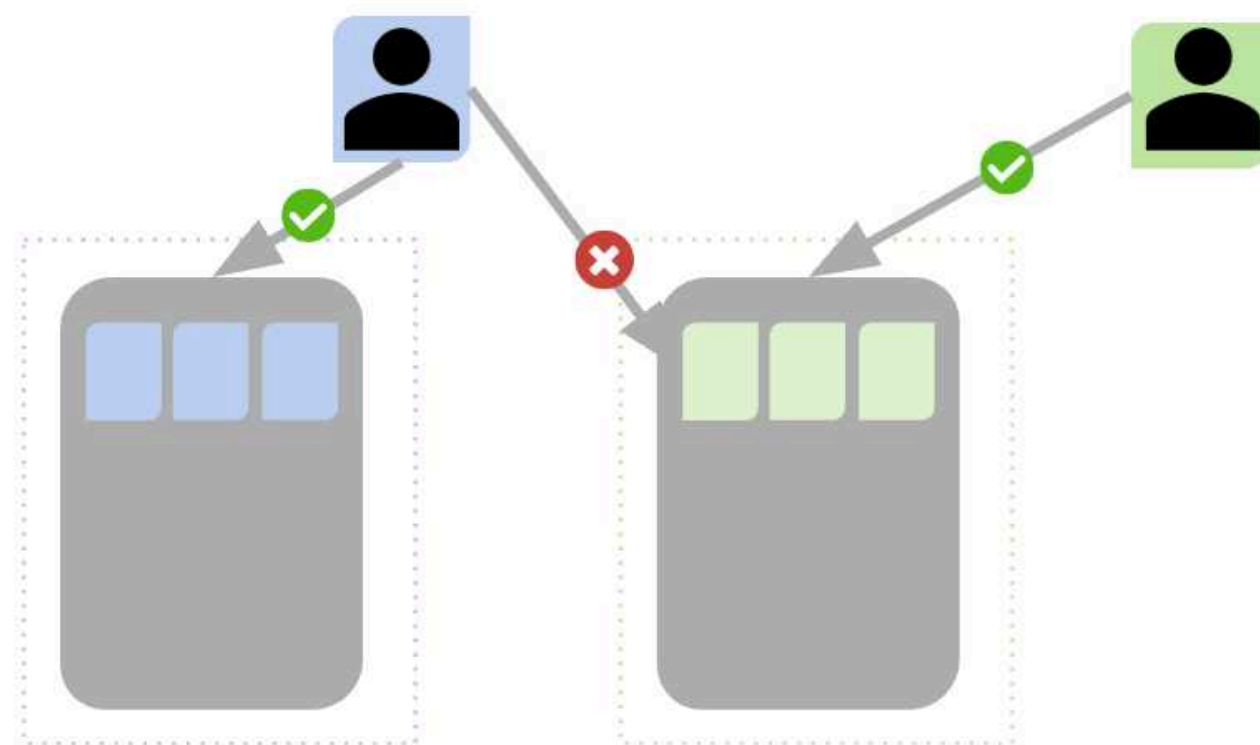




Control de acceso a la información de contexto

Sin un sistema de control de acceso basado en roles, cualquier usuario podrá realizar consultas sobre el índice vectorial (Index), ya que los vectores no distinguen entre roles ni privilegios de acceso. De este modo, aunque el usuario no tenga permiso para acceder al documento original, podría obtener respuestas relacionadas con él a través del sistema RAG.

Opción A: Una solución posible es incluir, como metadatos, los grupos de usuarios autorizados para acceder a cada documento, y durante la búsqueda por similitud de los fragmentos (chunks), aplicar un filtro que restrinja los resultados según esos grupos.



Opción A: Otra alternativa es segmentar la información en silos, por ejemplo, según departamentos o unidades, y construir varios índices independientes, asignando cada uno a un grupo específico de usuarios.

